AIVA

Project Implementation Report

Comprehensive summary of everything designed, built, tested, and verified

Document version: 1.0 | Date: 2026-08-26

Coverage: Milestones 0–11 (core platform) — delivered and CI-verified

Remaining: Milestone 12 (load test, penetration-test pass, retention jobs, Helm chart)

1. Executive Summary

AIVA is a complete, working, air-gapped AI-assisted hiring platform covering the full candidate lifecycle: requisition creation, resume ingestion and scoring, questionnaires, interview scheduling, a live adaptive interview with a sandboxed technical coding exercise, a retrieval-grounded FAQ, a cross-signal evaluation report, and a compliance/operations layer including GDPR/CCPA data-subject request tooling. Twelve milestones were planned; eleven are delivered and independently CI-verified against a live containerized stack, not merely unit-tested in isolation. The only remaining milestone is production hardening (load testing, a penetration-test pass, data-retention automation, and a Helm chart for Kubernetes deployment) — infrastructure work that depends on a target deployment environment this engagement does not yet have.

Two security reviews were run against the highest-risk additions during this engagement (the sandboxed code-execution service, and the M11 compliance/DSAR set). Both found real, concrete issues, which were fixed before being considered complete — not merely flagged. Details in Section 6.

2. Milestone-by-Milestone Summary

Milestone	What was delivered
M0 — Foundation	Monorepo scaffold, healthchecked Docker Compose stack (Postgres 16 + pgvector/pg_trgm, Redis 7, MinIO), fail-closed settings, strict CSP, egress-enforcement script, CI pipeline.
M1 — Design system	packages/ui: token-driven design system (colors, type scale, motion), theming, self-hosted variable fonts, shared component library consumed by both frontends.
M2 — Auth, RBAC, RLS, audit	Argon2id passwords, JWT access + rotating refresh tokens with reuse-detection, TOTP MFA, six-role RBAC, Postgres RLS FORCE on every org-scoped table, hash-chained audit log.
M3 — AI gateway	Stable typed contract for LLM judgements (rationale + confidence + cited evidence on every response), versioned prompt registry, deterministic mock backend, golden-set determinism harness.
M4 — Resume ingest, matching, scoring	Span-preserving PDF/DOCX/TXT extraction (every field resolves to page + offset + literal quote), deterministic weighted scoring with byte-identical run fingerprints, duplicate-resume rejection.
M5 — Recruiter console + Evidence Spine	Auth-gated pipeline UI, scored candidate cards, and a scroll-linked Evidence Spine showing every extracted field and judgement's literal source.
M6 — Questionnaires	Typed question schemas, single-use tokenized invites, resumable autosave with history, deterministic completeness enforcement.
M7 — Scheduling	DST-correct slot generation (verified against real spring-forward/fall-back edge cases), local RFC-5545 .ics generation, conflict-safe booking.

Milestone	What was delivered
M8 — Live interview sessions	Fail-closed consent → pre-check → active lifecycle, adaptive question engine driven by objective JD-vs-resume gaps, real candidate HUD with STT/TTS.
M9 — Live-coding workspace	Isolated code execution (per-run unprivileged Linux account, network+PID namespace isolation, resource limits), autosaved editor, shared whiteboard, discussion, deferred screen-share interface.
M10 — RAG FAQ + evaluation engine	Real pgvector retrieval behind a mock embedder, cross-signal deterministic evaluation aggregation, PDF/Excel report export.
M11 — Dashboard, blind screening, DSAR, and more	Org-wide pipeline dashboard, PII-redaction toggle, scoring-consistency audit, browser-based interview integrity signals, questionnaire cloning, full DSAR export/erasure.

Not started

M12 — Production hardening: load testing, a penetration-test pass, automated data-retention jobs, and a Helm chart for Kubernetes deployment. This is deliberately last: it requires a real target deployment environment and traffic profile to be meaningful, rather than a synthetic exercise against a development container.

3. Architecture

A monorepo of independently deployable services communicating over an internal network only, sharing one Postgres database whose Row-Level Security policies are the primary tenant-isolation boundary (not application-layer filtering alone).

Layer	Technology
Backend API	Python 3.11, FastAPI, SQLAlchemy 2.0 (async), Alembic (23 migrations)
Database	PostgreSQL 16 with pgvector + pg_trgm extensions
Cache / queue substrate	Redis 7 (AOF persistence)
Object storage	MinIO (S3-compatible)
AI serving	Internal ai-gateway microservice; mock-backed, real-model swap is deployment-time only
Code sandbox	Internal sandbox-runner microservice; process-level isolation, no shared kernel-adjacent state between runs
Frontends	React 18 + TypeScript (strict) + Vite; two apps (recruiter console, candidate journey)

Layer	Technology
Containerization	Docker Compose (dev); Kubernetes + Helm (production target, M12)

4. Project Metrics

Metric	Value
Backend (Python) lines of code	~8,000 across apps/api + services/ai-gateway + services/sandbox-runner
Frontend (TypeScript) lines of code	~4,100 across both web apps + packages/ui
Database migrations	12
Architecture Decision Records	23 (docs/DECISIONS.md)
Automated test functions	127+ across unit and live-stack integration suites
API routers	13 domain routers (auth, org, resume, questionnaire, scheduling, interview, workspace, faq, evaluation, dashboard, dsar, integrity, audit)
CI jobs	8 independent jobs (egress scan, compose validation, web lint/typecheck/build, three Python quality-gate jobs — api/gateway/sandbox, offline unit tests, one full live-stack integration job)

5. Testing and Quality Assurance

5.1 Test Strategy

- Pure-logic modules (scoring, scheduling, interview engine, evaluation engine, scoring audit, sandbox executors) are unit-tested directly with no database or network dependency.
- Every domain lifecycle (auth, resume, questionnaire, interview, live-coding workspace, FAQ, evaluation, and the full M11 set) is proven end-to-end against a live containerized stack — real Postgres with RLS enforced, a real AI gateway container, a real sandbox-runner container — not a mocked database layer.
- Determinism is directly tested, not assumed: identical resume-scoring and mock-AI-gateway inputs are asserted to produce byte-identical outputs across repeated calls.
- Isolation properties are directly tested, not assumed: a live socket-connect attempt from inside the code sandbox is confirmed blocked by the network namespace; the per-run uid pool is confirmed to never hand out the same uid to two concurrent executions; PID-namespace isolation is confirmed by reading back the sandboxed process's own PID.

5.2 Quality Gates (all green as of this report)

- ruff, black --check, mypy --strict, and bandit across apps/api, services/ai-gateway, and services/sandbox-runner

- ESLint (typescript-eslint, no `any`, no non-null assertions) and strict TypeScript typechecking across both frontends
- A full production build of both frontends
- A repository-wide external-URL scan, negative-tested (a planted violation is confirmed to fail the scan before being removed)

6. Security Review Findings and Fixes

Two independent, targeted security reviews were run during this engagement against the highest-risk diffs, following the same methodology: identify candidate vulnerabilities, then independently re-verify each one against the actual code and this project's established patterns before accepting it, filtering out anything below high confidence.

6.1 Sandboxed code execution (M9)

Finding (High): the first implementation setuid-dropped every candidate code execution into the same fixed unprivileged account. Because every concurrent run shared that account, one candidate's sandboxed code could list the shared temp directory, discover another live run's working directory (owned by the identical uid), and read or overwrite it — a real cross-session (and potentially cross-organization) data leak reachable through the platform's own normal "run code" API, with no special tooling required.

Fix: a per-run UID pool (never shared between concurrent executions, blocking rather than reusing a uid under load) plus PID-namespace isolation, so two concurrent runs can neither share filesystem ownership nor see or signal each other via /proc. Both properties are now directly unit-tested. Recorded as ADR-020.

6.2 DSAR erasure completeness (M11)

Finding (High): the first implementation of candidate-data erasure redacted `ResumeDocument.full_text`/`.candidate_email` and `ExtractedFieldRow.value`, but left two sibling columns on those same rows untouched: `ResumeDocument.filename` (commonly identity-revealing, e.g. `jane_doe_resume.pdf`) and `ExtractedFieldRow.source_quote` (the raw resume text surrounding a matched PII span, so routinely contains the literal email/phone/name in context). Both were still readable through the platform's ordinary, non-blind resume-detail endpoint after an admin had already run and confirmed an erasure request — a GDPR/CCPA erasure feature that did not fully erase.

Finding (Medium): the companion export endpoint accepted the candidate's raw email as a GET query parameter, inconsistent with the erasure endpoint's own POST-body design in the same file — and a query string is exactly what reverse-proxy and application access logs capture by default, undermining the module's own discipline of only ever persisting a SHA-256 hash of the email to the audit log.

Fix: both gaps closed — filename and source_quote are now redacted alongside the other PII fields, and export was switched to a POST body matching the erase endpoint's shape. Both fixes are directly asserted by the integration test suite, re-verified against the live stack after the change.

6.3 Dashboard aggregate query (M11)

Bug (functional, caught in live-stack verification): the recruiter dashboard's coding-task pass-rate query used `max(uuid)` to identify each task's most recent execution — PostgreSQL has no such aggregate function for the uuid type, so the endpoint failed outright against a live database. Fixed by joining on `(task_id, created_at)` instead, which is also what "most recent" actually means. Caught only because the endpoint was exercised against a real Postgres instance in the integration test suite, not a mocked one.

6.4 Resume detail page (pre-existing, found during M10 verification)

Bug (functional, pre-existing, unrelated to the diff being verified): screenshot-testing the new evaluation panel surfaced a crash in the recruiter's resume detail page on any resume that had a scoring run: the list endpoint for scoring runs never returned the `checks`/`dimensions` fields, but the page's "latest run" selector assumed they were present. Fixed forward rather than left for a future milestone to rediscover, since the same live-stack verification step that found M10's own issues also caught it.

6.5 Documentation drift

The AI model inventory (docs/MODEL_CARD.md) still described several capabilities that had since shipped (M3's LLM gateway, M10's embeddings) as "pending," and named a different candidate embedding model than what was actually built. Corrected during this report's preparation, since a stale model inventory is itself a governance risk for a project whose compliance story depends on accurate documentation.

7. Known Limitations (Documented, Not Hidden)

- **Real AI model inference** (LLM reasoning/scoring, STT, TTS, embeddings) is deferred to GPU hardware deployment; every capability is built and CI-verified against a deterministic mock backend behind a stable interface, per docs/MODEL_CARD.md.
- **Screen sharing** (M9) has a stable API contract but no WebRTC/LiveKit backend; the endpoint returns a clear 501 rather than a false success.
- **Face/gaze-based interview proctoring** (InsightFace/MediaPipe) remains deferred to GPU deployment; M11 shipped a zero-ML alternative (browser tab-focus/visibility signals) rather than an unbacked mock, by deliberate decision (ADR-023).
- **Demographic bias auditing** is not implemented: this system collects no protected-characteristic data about candidates, so a disparate-impact analysis is not implementable responsibly. M11's "scoring audit" instead verifies the scoring pipeline's own internal consistency guarantees (ADR-023).
- **DSAR erasure** overwrites top-level PII fields but does not deep-redact resume quotes embedded inside JSONB evidence payloads (scoring checks/dimensions) — a documented gap, not a silent one (ADR-022).
- **Two narrower test-coverage gaps** carried forward from earlier milestones: the resume candidates-list endpoint (M5) has no dedicated automated test, and interview scheduling (M7) has unit but not live-stack integration coverage. Both areas function correctly under manual and unit testing; only the automated safety net is narrower than the rest of the platform.

8. Recommendations / Next Steps

1. Deploy GPU hardware and swap the mock LLM/STT/TTS/embedding backends for the real models named in docs/MODEL_CARD.md — no application code changes are required for this swap.
2. Begin Milestone 12 once a target production environment is selected: load testing against realistic traffic, a third-party penetration-test pass, automated data-retention jobs, and a Helm chart for the chosen Kubernetes environment.
3. Close the two narrower test-coverage gaps noted in Section 7 (candidates-list endpoint test, scheduling live-stack integration test).
4. Decide, as a product/legal question rather than an engineering one, whether this system should ever collect protected-characteristic data for a true disparate-impact bias audit; the current scoring-consistency audit is a deliberate, honest substitute, not a placeholder for that decision.

End of Project Implementation Report.